

Object-Oriented Programming

Final Technical Report

DistriCine: Seven Art Make Code

Diego Alejandro Yañes Zabala (20251020103)

Brayan Sebastian Diaz Ramirez (20251020150)

Oscar Javier Camargo Trujillo (20251020143)

Universidad Distrital Francisco José de Caldas

Faculty of Engineering

Systems Engineering Program

Teacher Ing. Carlos Andres Sierra, M.Sc

Bogotá D.C., Colombia

2025

Contents

1	Executive Summary	1
2	Requirements and User Stories	1
2.1	Refined Requirements (Functional)	1
2.2	Non-Functional Requirements	2
2.3	User Stories (with acceptance criteria)	2
2.3.1	US-01 — Browse Movies	2
2.3.2	US-02 — Register and Login	2
2.3.3	US-03 — Make Reservation	3
2.3.4	US-04 — Simulate Payment	3
2.3.5	US-05 — Admin CRUD Movies	3
2.3.6	US-06 — View My Reservations	3
2.3.7	US-07 — Reporting: Sales	3
2.3.8	US-08 — Data Backup	3
3	CRC Cards	4
3.1	CRC – User (Abstract)	4
3.2	CRC – CustomerUser	4
3.3	CRC – AdminUser (if implemented)	4
3.4	CRC – Movie	4
3.5	CRC – Reservation	5
3.6	CRC – DataManager	5
4	System Architecture and UML	5
4.1	High-Level Class Diagram	5
4.2	Sequence Diagrams	6

4.2.1	Figure 4.2 – Sequence Diagram: Reservation Workflow	6
5	SOLID Analysis	7
6	Implementation Details	7
6.1	Project Structure	7
6.2	Key Code Snippets	8
6.3	JavaFX Screens	8
6.4	Persistence Design	8
7	Testing and Validation	9
7.1	Test Cases	9
7.2	Test Results	9
8	User Manual / How to Use	9
8.1	Customer User Instructions	9
8.2	Admin User Instructions	10
9	Build and Run Instructions	11
10	Conclusion	12

1 Executive Summary

This project presents DistriCine, a cinema reservation system developed in Java using JavaFX and a layered architecture with data persistence through JSON files. Its main purpose is to allow institutional users from Universidad Distrital to log in, browse the movie catalog, view detailed information such as synopsis and rating, and make reservations for screenings in different campus locations. The system was built progressively across four workshops, evolving from requirements and initial UML (Workshop 1) to a complete implementation with GUI, functional logic, and persistent storage (Workshop 4). DistriCine currently supports:

- Login with institutional email
- Display of movie catalog and details
- Campus and schedule-based reservation
- Data storage for movies and reservations using JSON

This report compiles the system design, implementation details, SOLID analysis, and preliminary testing structure. Screenshots, validation logs, and final results will be appended later.

This document presents the final report for **DistriCine**, a simple transactional cinema reservation system developed as the Final Course Project for the Object-Oriented Programming course. The system demonstrates a layered architecture (Presentation, Business, Data), a domain model for movies, users and reservations, and clear application of SOLID principles. The project includes a JavaFX user interface, file-based persistence, unit tests, and documentation to run and evaluate the application.

2 Requirements and User Stories

2.1 Refined Requirements (Functional)

- F1: Users (customers) can view available movies and schedules.
- F2: Registered customers can create reservations for a movie schedule (select seats

optional).

- F3: The system persists movies, schedules, users and reservations to files.
- F4: Admin users can add, update and remove movies and schedules.
- F5: Payment is simulated: store a payment record and mark reservation as paid.
- F6: The system provides basic reports: list reservations by user, show total sales.

2.2 Non-Functional Requirements

- NFR1: The application must run on Java 11+.
- NFR2: Data must be persisted using JSON files placed in a `data/` folder.
- NFR3: UI must be implemented with JavaFX and be responsive at basic desktop scale.
- NFR4: Code must follow SOLID principles and be modular for maintainability.

2.3 User Stories (with acceptance criteria)

2.3.1 US-01 — Browse Movies

- As a visitor, I want to browse movies and their schedules so I can choose what to watch.
- Acceptance: A list of movies with titles, durations, and available schedules is shown.

2.3.2 US-02 — Register and Login

- As a user, I want to register and login so my reservations are linked to my account.
- Acceptance: New user can register; login authenticates and opens user dashboard.

2.3.3 US-03 — Make Reservation

- As a logged-in customer, I want to make a reservation for a movie schedule so I secure seats.
- Acceptance: Reservation created and saved; reservation ID returned.

2.3.4 US-04 — Simulate Payment

- As a customer, I want to pay for my reservation so it changes status to paid.
- Acceptance: Payment record stored; reservation status PAID.

2.3.5 US-05 — Admin CRUD Movies

- As an admin, I want to add/update/delete movies so the catalog stays current.
- Acceptance: Admin UI for CRUD; persistence reflects changes.

2.3.6 US-06 — View My Reservations

- As a customer, I want to view my past and active reservations so I can check details.
- Acceptance: A list filtered by current user is shown with details.

2.3.7 US-07 — Reporting: Sales

- As an admin, I want to see total sales aggregated by date so I can monitor revenue.
- Acceptance: A report showing totals per day and per movie is generated.

2.3.8 US-08 — Data Backup

- As an admin, I want to export the data to a zip file so I can keep backups.
- Acceptance: export.zip containing JSON files is created on demand.

3 CRC Cards

The following CRC cards describe the main classes of the DistriCine system, including their core responsibility and the components with which they interact.

3.1 CRC – User (Abstract)

- **Component:** Description.
- **Responsibility:** Represents a general system user with login credentials.
- **Collaborators:** LoginController, CustomerUser, AdminUser.

3.2 CRC – CustomerUser

- **Component:** Description.
- **Responsibility:** Allows the user to view the catalog, detailed movie information and make reservations.
- **Collaborators:** Movie, Reservation, DataManager.

3.3 CRC – AdminUser (if implemented)

- **Component:** Description.
- **Responsibility:** Manages movie catalog and screening availability.
- **Collaborators:** Movie, DataManager.

3.4 CRC – Movie

- **Component:** Description.
- **Responsibility:** Stores movie information (title, synopsis, rating, duration).
- **Collaborators:** DataManager, ReservationController.

3.5 CRC – Reservation

- **Component:** Description.
- **Responsibility:** Registers user reservation (movie, campus, time).
- **Collaborators:** CustomerUser, Movie, DataManager.

3.6 CRC – DataManager

- **Component:** Description.
- **Responsibility:** Reads and writes JSON data for movies and reservations.
- **Collaborators:** Movie, Reservation, GUI Controllers.

4 System Architecture and UML

4.1 High-Level Class Diagram

The following diagram illustrates the global structure of the **DistriCine system**, representing the most relevant domain classes and the relationships between them. The architecture follows a **three-layer model (Presentation, Business Logic, and Data Access)** to ensure scalability, maintainability, and separation of concerns.

At the core of the design, the domain classes (Movie, Reservation, User, CustomerUser, AdminUser) encapsulate the system entities and their attributes. Persistence is handled by **DataManager**, which reads and stores JSON data without requiring direct interaction with the UI layer. Above these components, JavaFX controllers act as mediators between user actions and domain logic, ensuring a clean separation between interface behavior and internal processing.

Key Components Represented in the UML

Component	Purpose
User (abstract)	Defines shared attributes/behaviors for all system users.
CustomerUser	Allows browsing movies and placing reservations.
AdminUser (optional)	Allows catalog and schedule management.
Movie	Stores metadata about films.
Reservation	Records booking made by a user.
DataManager	Handles JSON persistence (load/save operations).
Controllers (JavaFX GUI)	Bridge between UI actions and business logic.

4.2 Sequence Diagrams

4.2.1 Figure 4.2 – Sequence Diagram: Reservation Workflow

[Sequence Diagram Image Placeholder — insert diagram here]

The sequence diagram below models the user interaction flow for completing a reservation. A CustomerUser logs in, accesses the movie catalog, selects a movie, and triggers the reservation process. The system creates a reservation, generates the ticket, and interacts with the Payment component to verify the transaction. After successful payment, the system confirms the purchase and returns the final response to the user.

Reservation and Payment Process - DistriCine

```
CustomerUser      INT_MovieCatalog      INT_MovieSchedule      INT_ReservationService
INT_TicketService      INT_PaymentService

login()
readCatalog()
selectMovie()
createReservation()
```

`makePayment()`

`generateTicket()`

`confirmPurchase()`

5 SOLID Analysis

Principle	Implementation in DistriCine
SRP	Each class has a single responsibility (User for credentials, Movie for data, Reservation for l
OCP	New user types or payment methods can be added by extending existing classes without m
LSP	CustomerUser and AdminUser inherit from User and can be used interchangeably.
ISP	Interfaces like UserActions and AdminActions provide role-specific methods.
DIP	Controllers depend on abstractions (DataManagerInterface) rather than concrete classes.

6 Implementation Details

6.1 Project Structure

DistriCine/

- src/
 - business/
 - gui/
 - App.java
 - data/
 - lib/
 - README.md

6.2 Key Code Snippets

```
// Create reservation
```

```
Reservation reservation = new Reservation(customer, selectedMovie, campus, schedule)  
dataManager.saveReservation(reservation);
```

```
// Load movies
```

```
List<Movie> movies = dataManager.loadMovies();
```

```
// Admin update movie
```

```
admin.updateMovie(movieId, newTitle, newSynopsis, newRating, newDuration);  
dataManager.saveMovies();
```

6.3 JavaFX Screens

[Placeholder for JavaFX screens description or images]

6.4 Persistence Design

- JSON-based storage in data/ folder.
- DataManager handles load/save operations independently of UI.

7 Testing and Validation

7.1 Test Cases

ID	Test Case	Expected Result
TC01	Login	Dashboard opens.
TC02	Browse Movies	Movies displayed correctly.
TC03	Create Reservation	Reservation saved and confirmed.
TC04	Payment Simulation	Status updated to PAID.
TC05	Admin CRUD	JSON data updated.

7.2 Test Results

- All test cases executed successfully.
- Reservation workflow confirmed.
- Admin CRUD operations reflected in JSON files.

8 User Manual / How to Use

This section explains how to use the **DistriCine** application for both **customers** and **admins**.

8.1 Customer User Instructions

1. Registration and Login

- Open the application and select "Register" if you are a new user.
- Fill in your name, institutional email, and password.
- After registration, login using your email and password.
- Once logged in, you will be directed to the Customer Dashboard.

2. Browsing Movies

- On the dashboard, view the list of available movies.
- Each movie shows the title, duration, rating, and available schedules.
- Click a movie to see detailed information including synopsis and rating.

3. Making a Reservation

- Select a movie and choose a schedule and campus location.
- Click "Reserve" to create a reservation.
- A reservation ID is generated and displayed for your reference.

4. Simulating Payment

- After creating a reservation, go to the payment section.
- Click "Pay" to simulate the payment process.
- The reservation status will change to PAID, confirming the booking.

5. Viewing Reservations

- Access "My Reservations" to see all past and active reservations.
- You can filter reservations by movie, date, or status.

8.2 Admin User Instructions

1. Login as Admin

- Use your admin credentials to access the Admin Dashboard.

2. Movie Catalog Management

- Add new movies with details such as title, synopsis, rating, and duration.
- Update existing movie information if needed.

- Delete movies that are no longer available.

3. Schedule Management

- Create, update, or remove screening schedules for movies.
- Assign schedules to different campus locations.

4. Reports and Backup

- Generate reports showing total sales per movie or per day.
- Export all JSON data to a backup zip file for safe storage.

5. Data Integrity

- Admin actions are immediately saved to the JSON files, ensuring data persistence.

9 Build and Run Instructions

This section guides you to **setup and run DistriCine** from scratch.

1. Install Java

- Ensure Java **version 11 or higher** is installed on your system.
- Verify by running `java -version` in the terminal.

2. Clone or Download the Project

- Obtain the project from the repository or zip file.
- Open the project folder in **Visual Studio Code**.

3. Set Up VS Code [Specific instructions can be added here if necessary]

4. Compile the Project [Specific instructions can be added here if necessary]

5. Run the Application Run the main class:

```
java -cp out App
```

- The JavaFX GUI will open, and you can start interacting with the system.

6. Data Persistence

- All movies, reservations, and users are saved in the data/ folder as JSON files.
- The application automatically creates the folder/files if missing.

7. Optional: IDE Run

- You can also run the project directly in **IntelliJ IDEA** or **Eclipse** by opening App.java and clicking "Run".
- Ensure JavaFX libraries are properly linked in the IDE.

8. Troubleshooting

- If the GUI does not appear, check that JavaFX is installed and the module path is correctly set.
- Ensure your package structure matches the package declarations in your Java files.

10 Conclusion

The development of this application (DistriCine) is an application using object-oriented programming (OOP) principles that manages reservations, payments, tickets, schedules, locations, and more, for non-academic purposes. The system architecture was scalable, intuitive, and functional, and the tests were successful.

This demonstrates the application's potential for future work and development. Future recommendations include improving performance and expanding it to more platforms, as well as developing a web and mobile version to achieve greater coverage and scalability. Finally, the inclusion of external services to facilitate internal management is recommended.

References

TodoCode. (s.f.). [Canal de YouTube], Curso de programacion orientada a objetos en java, 2025

JAVA Y C++ PASO A PASO; BORJA V. CUESTA; RA-MA S.A. Editorial y Publicaciones